

Trabajo Práctico Final

Arquitectura de Computadoras y Sistemas Embebidos

Autor: Diego Márquez

Profesores: Leonardo Enrique Castillo, Rigoberto Acosta González

Fecha de entrega: 23 de junio de 2026

Índice general

Índice general	I
1. Planteo y Evolución Conceptual	1
2. Armado y Programación	2
1. Materiales	2
2. Desarrollo y Configuración	2
2.1. FFT	2
2.2. Lectura y Envío de Datos	2
2.3. Access Point y Página Web	4
2.4. Límites de Energía	4
3. Discusión - Conclusiones	5
4. Anexo	6
1. Anexo A: Repositorio GitHub	6

Capítulo 1

Planteo y Evolución Conceptual

Originalmente se planteó la idea de programar un vúmetro, utilizando 8 LEDs, para medir la intensidad del sonido mediante un micrófono y el ADC del ESP32C3. Sin embargo esto se descartó por el exceso de simplicidad que implicaba.

Al considerar interesante el planteo original, se optó por complejizarlo, manteniendo esencialmente el concepto, pero añadiendo una serie de funciones que complicarían su desarrollo y ejecución.

- Se decidió optar por un híbrido entre vúmetro y analizador de espectros, así, este reaccionaría a la energía presente en el sonido en función de su frecuencia. Un ecualizador, en definitiva.
- En lugar de utilizar una matriz LED de 8x8, se optó por diseñar el programa considerando 56 LEDs individuales (7 niveles de frecuencia, 8 niveles de energía). Estos se controlan utilizando 7 desplazadores de registro de 8 bits (configuración de 7 bandas, común en ecualizadores).
- Se añadió la implementación de una página web que muestra las mediciones en forma gráfica (barras), coincidiendo con lo visible en los LEDs.

En base a esto, se desarrolló el proyecto dentro de las limitaciones materiales de la universidad (componentes).

Capítulo 2

Armado y Programación

1. Materiales

Para el desarrollo de este proyecto se necesitaron los materiales listados a continuación:

- Protoboard.
- 56 diodos LED.
- 7 desplazadores de registro *TC74HC595*.
- Micrófono con salida de datos analógica
- Microcontrolador ESP32C3 Super Mini

2. Desarrollo y Configuración

2.1. FFT

Luego de determinar la idea final del proyecto, se buscó la configuración típica de un ecualizador de 7 bandas, sus frecuencias estandarizadas (63 Hz, 160 Hz, 400 Hz, 1 kHz, 2.5 kHz, 6.3 kHz y 16 kHz). A partir de estos datos, se pudo determinar una frecuencia de muestreo que debía ser el doble o mayor de la frecuencia máxima (por Teorema de Nyquist), por lo que se optó por establecer la misma en 32 kHz. Luego, se optó por un número de muestras que debía ser potencia de 2, en este caso, se eligió 512.

Luego, se configuró una FFT para convertir la señal del dominio del tiempo al de la frecuencia, obteniendo así la energía en los graves, medios y agudos, que determina la cantidad de LEDs encendidos en cada banda. Se optó por el método Cooley-Tukey, porque este reduce drásticamente la complejidad computacional al compararlo con la Transformada de Fourier Discreta convencional (de $O(N^2)$ a $O(N \times \log_2(N))$). Adicionalmente, se añadió una función que calcula la magnitud de la señal, luego de igualar la parte imaginaria de la señal a 0, para poder determinar adecuadamente la energía de la señal y así la cantidad de LEDs a prender y qué tanto suben las barras en la página.

2.2. Lectura y Envío de Datos

Luego, se desarrolló una función que envía los datos procesados hacia los desplazadores de registros 74HC595 que, mediante el enmascarado de bytes, extrae bits específicos. En esta función se implementaron delays de $1\mu s$ para que la velocidad de envío de datos no sobrepase la capacidad de los desplazadores.

```
//Función para enviar datos al desplazador de registros
void enviar74HC595(uint8_t *datos, int numColumnas){
    for(int i=numColumnas-1; i>=0; i--){
        uint8_t auxDatos=datos[i];
```

```

        for(int j=7; j>=0; j--){
            gpio_set_level(pinDatos, (auxDatos>>j)&1);
            esp_rom_delay_us(1);
            gpio_set_level(pinClock, 1);
            esp_rom_delay_us(1);
            gpio_set_level(pinClock, 0);
            esp_rom_delay_us(1);
        }
    }
    gpio_set_level(pinLatch, 1);
    esp_rom_delay_us(1);
    gpio_set_level(pinLatch, 0);
}

```

Se optó por usar el ADC continuo (DMA), en lugar del oneshot que se había empleado en trabajos anteriores, esto se hizo más que nada para probar su funcionamiento, el cual lee el voltaje del micrófono a 32 kHz en segundo plano y lo almacena en RAM, sin gastar ciclos de la CPU. Además, para una correcta lectura del micrófono, se hizo una función que remueve el voltaje de offset, calculando el promedio de la onda y restándolo en cada punto, para centrar el audio en cero.

```

//Función remueve offset DC
void removerCC(float *datos, uint16_t muestras){
    float media=0;
    for(uint16_t i=0; i<muestras; i++){
        media+=datos[i];
    }
    media/=muestras;
    for(uint16_t i=0; i<muestras; i++){
        datos[i]-=media;
    }
}

//Inicializo ADC en Modo Continuo (DMA)
void iniciarADC(){
    adc_continuous_handle_cfg_t cfg={
        .max_store_buf_size=numMuestras*SOC_ADC_DIGI_RESULT_BYTES*4,
        .conv_frame_size=numMuestras*SOC_ADC_DIGI_RESULT_BYTES,
    };
    adc_continuous_new_handle(&cfg, &adc_handle);
    adc_digi_pattern_config_t patron={
        .atten=ADC_ATTEN_DB_12,
        .channel=ADC_CHANNEL_0,
        .unit=ADC_UNIT_1,
        .bit_width=ADC_BITWIDTH_12,
    };
    adc_continuous_config_t dig_cfg={
        .sample_freq_hz=frecuenciaMuestreo,
        .conv_mode=ADC_CONV_SINGLE_UNIT_1,
        .format=ADC_DIGI_OUTPUT_FORMAT_TYPE2,
        .pattern_num=1,
        .adc_pattern=&patron,
    };
    adc_continuous_config(adc_handle, &dig_cfg);
    adc_continuous_start(adc_handle);
}

```

2.3. Access Point y Página Web

Para armar el punto de acceso del microcontrolador junto con la página web, se empezó por configurar WebSocket para que la página pueda funcionar acorde al protocolo HTTP, para poder enviar matrices de texto (barras para niveles de bandas). La parte más importante de todo esto fue que el funcionamiento sea asíncrono, pues de no serlo, el bucle principal se congelaría esperando la red por los tiempos que maneja.

```
//Envío a la página web
if (ws_client_fd!=-1 && global_server!=NULL){
    static char datos_ws[32];
    //Arreglo [2, 8, 0...] a texto "2,8,0..."
    snprintf(datos_ws, sizeof(datos_ws), "%d,%d,%d,%d,%d,%d,%d", nivelesLED[0],
nivelesLED[1], nivelesLED[2], nivelesLED[3], nivelesLED[4], nivelesLED[5],
nivelesLED[6]);
    httpd_ws_frame_t ws_pkt;
    memset(&ws_pkt, 0, sizeof(httpd_ws_frame_t));
    ws_pkt.payload=(uint8_t*)datos_ws;
    ws_pkt.len=strlen(datos_ws);
    ws_pkt.type=HTTPD_WS_TYPE_TEXT;
    esp_err_t err_ws = httpd_ws_send_frame_async(global_server, ws_client_fd, &ws_pkt);
    if (err_ws != ESP_OK) {
        ws_client_fd = -1;
    }
}
```

La configuración del punto de acceso no fue muy distinta de los trabajos anteriores, se estableció el nombre de la red, contraseña, tipo de encriptado, etc. Además, se añadió una variable a modo de contador implementando *Throttling* para refrescar la red, de forma tal que el WiFi no asfixie a la CPU y dispare el Watchdog.

2.4. Límites de Energía

En el bucle principal, se optó por establecer variables que determinan el máximo de energía (valor leído para el cual todos los LEDs se prenden y las barras llegan al tope), y el mínimo de energía (valor leído para el cual los LEDs quedan apagados y las barras vacías). El motivo principal de esto fue la calidad del micrófono empleado. Estableciendo estos valores, se puede ajustar su sensibilidad vía software en lugar de vía hardware, adecuándolo al entorno independientemente de sus características.

Capítulo 3

Discusión - Conclusiones

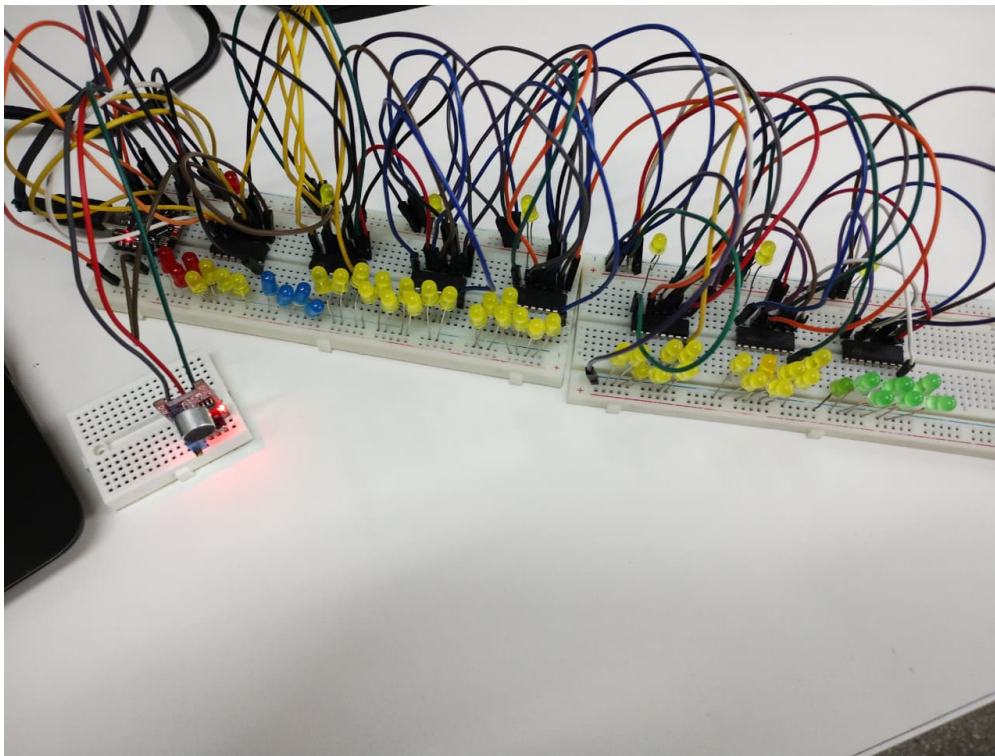


Figura 3.1: Circuito Montado

En este trabajo se emplearon prácticamente todas las herramientas vistas durante la cursada, al menos las usadas en este microcontrolador.

El proyecto como tal podría haber sido realizado de manera mucho más sencilla y eficiente (en términos de tiempo de trabajo) si se hubiera usado MicroPython, esto porque la gran mayoría de funciones y arreglos hechos en el código presentado se hicieron justamente porque no existen equivalencias en C. Sin embargo, al menos en mi caso, C es un lenguaje al que estoy mucho más acostumbrado, tanto para leerlo como para programar en él. Por esto se eligió este último lenguaje.

Otra consideración podría ser emplear una matriz LED, o una serie de tiras LED, en lugar de 56 LEDs individuales con desplazadores de registros. No obstante, frente a la falta de tiras en el laboratorio y el hecho de que las matrices LED estaban prácticamente en su totalidad quemadas parcialmente (columnas y filas que no encendían), sostengo que la alternativa elegida fue la correcta, aún frente a lo aparatoso que pueda resultar ensamblar el circuito en protoboards.

Capítulo 4

Anexo

1. Anexo A: Repositorio GitHub

`https://github.com/DiegoMrqzz/AdCySE-Trabajo\_Final`